

ENTREGA SEQUENCIAL 4

Atualização do Passo a Passo Explicativo para a Implementação do Bloco 1 (Atualizado)

(ver observações no final do texto que resultaram essa atualização)

Passo a passo detalhado em linguagem natural de como construir para implementar o Bloco 1:

- **Passo 1: Importar a ferramenta de datas**
 - **O que fazer:** Logo no início do arquivo, importe apenas a funcionalidade `date` da biblioteca `datetime`.
 - **Por quê:** Ela vai permitir criar e manipular datas de compra e vencimento sem precisar incluir horas e minutos no controle de inventário.
- **Passo 2: Criar a classe mãe Produto**
 - **O que fazer:** Defina a classe base chamada `Produto`. No método de inicialização (`__init__`), declare três parâmetros: o código do produto (`id_produto`), o nome (`nome`) e o preço de varejo (`preco_venda`). Implemente também o método especial `__repr__` para que as informações do produto sejam impressas de forma limpa e legível durante os testes.
 - **Por quê:** Todo item vendável no restaurante compartilha essas três características fundamentais, e o termo "preço de venda" diferencia o valor de cardápio do custo pago aos fornecedores.
- **Passo 3: Criar as classes filhas Comida e Bebida (Herança)**
 - **O que fazer:**
 1. Crie a classe `Comida` herdando de `Produto`. Na inicialização, receba o parâmetro `peso_g` e use a instrução `super().__init__()` para repassar o id, nome e preço de venda para a classe mãe.
 2. Crie a classe `Bebida` herdando de `Produto`. Na inicialização, receba a categoria (ex: "Água", "Suco", "Refrigerante"). Ao chamar o `super().__init__()`, passe a própria categoria como o nome do produto. Defina o atributo `volume_ml` diretamente como 500 por padrão dentro da classe.
 - **Por quê:** Isso atende fielmente à premissa do negócio de que as bebidas têm volume padronizado em 500ml e preços fixados pela sua categoria.

- **Passo 4: Criar a classe LoteEstoque**
 - **O que fazer:** Crie uma classe que receba e guarde cinco informações essenciais: a referência do objeto produto, a quantidade de itens, o custo de aquisição (`preco_compra`), a `data_compra` e a `data_vencimento` (ambas utilizando objetos `date`). Adicione o método `__repr__` para detalhar o lote no terminal.
 - **Por quê:** Para o controle de estoque funcionar, **você precisa rastrear o preço pago ao fornecedor diretamente em cada remessa, já que os custos de mercado flutuam, enquanto o preço de venda no cardápio continua estável.**
- **Passo 5: Criar a classe Estoque e a lista de controle**
 - **O que fazer:** Crie a classe `Estoque`. No método de inicialização (`__init__`), declare uma lista vazia chamada `self.lotes`.
 - **Por quê:** Essa lista **servirá como o repositório central onde todos os lotes que chegarem serão armazenados e processados digitalmente.**
- **Passo 6: Implementar a inserção com ordenação cronológica e a edição manual**
 - **O que fazer:**
 1. Crie o método `adicionar_lote(self, lote)`. Dentro dele, **adicione o lote recebido à lista `self.lotes` com `.append()` e, logo em seguida, utilize a função `.sort(key=lambda l: l.data_compra)` para reorganizar toda a lista com base na data em que foram comprados.**
 2. Crie o método `editar_quantidade(self, lote, nova_quantidade)` para permitir que o usuário altere manualmente o saldo de um lote específico, impedindo apenas valores negativos.
 - **Por quê:** **A ordenação automática impede que falhas na ordem de digitação das notas fiscais quebrem a lógica do FIFO.** A edição atende à obrigatoriedade do enunciado de permitir correções manuais no inventário.
- **Passo 7: Implementar a lógica de consulta e a baixa PEPS/FIFO Segura**
 - **O que fazer:**
 1. Crie um método auxiliar chamado `consultar_quantidade_total(self, id_produto)` que varre os lotes e faz a soma matemática de todas as unidades disponíveis daquele item.
 2. Crie o método principal `dar_baixa_fifo(self, id_produto, quantidade_desejada)`: Primeiro, compare a quantidade solicitada com o retorno da consulta total. Se o saldo for insuficiente, exiba um erro em tela e aborte imediatamente a função retornando `False`. Caso haja

saldo, crie uma variável `qtd_restante` e execute um laço `for` pelos lotes. Se o lote for do produto correto e tiver saldo, consuma os itens sequencialmente até suprir a necessidade (`qtd_restante == 0`) e encerre com `break`. Ao concluir com sucesso, retorne `True`.

- **Por quê:** Essa estrutura em duas etapas impede o bug da "falsa baixa". Se o operador do caixa cometer um erro de digitação e pedir mais do que o restaurante possui, o sistema bloqueia a transação antes de alterar qualquer propriedade dos lotes, mantendo a consistência absoluta dos dados.

Observação 1:

1)O código fornecido para o **Bloco 1** está estruturalmente bom, bem documentado e segue corretamente os princípios de Programação Orientada a Objetos (POO) com herança e encapsulamento.

No entanto, para que o sistema funcione perfeitamente de acordo com as regras de negócio descritas no enunciado, a lógica do método `dar_baixa_fifo` precisa de um ajuste crítico de **segurança de dados**.

O Problema Atual: "Falsa Baixa"

Do jeito que o código está escrito, se o usuário tentar retirar uma quantidade maior do que a disponível no estoque (ex: pedir 10 itens e só ter 4), o sistema vai rodar o laço, **zerar o estoque dos lotes existentes** e, só no final, exibir a mensagem de erro.

Isso gera um problema grave: o estoque é consumido incorretamente e a venda não é concluída com sucesso, corrompendo os dados do sistema.

Observação 2:

Há três pontos que precisam ser corrigidos:

1. Falta o preço de compra. O enunciado pede que a estrutura de estoque guarde nome do produto, preço de compra, preço de venda, data da compra, data de vencimento e quantidade. O código atual só tem um `preco` dentro de `Produto`, e o `LoteEstoque` não guarda nenhum preço. Isso é **um requisito explícito do enunciado que ainda não foi atendido**.

A solução mais correta, do ponto de vista de negócio, é separar os dois. O preço de venda é uma característica do produto em si, já o preço de compra varia a cada remessa, então ele pertence ao lote, não ao produto.

2. Falta o método de editar quantidade. O item "b" do Controle de Estoque pede uma forma de editar a quantidade em estoque. Hoje só existe `adicionar_lote` e a baixa via FIFO, não há como corrigir manualmente uma quantidade.

3. O FIFO depende da ordem de inserção, não da data. `dar_baixa_fifo` assume que o primeiro lote da lista é sempre o mais antigo. Isso só é verdade se `adicionar_lote` for sempre chamado na ordem cronológica de compra. Vale deixar isso explícito ou garantir de forma automática.